PERTEMUAN 12

Konsep Dasar Pengujian Perangkat Lunak

Topik

- **Konsep dasar pengujian PL**
- **Pengujian tahap pengembangan PL**
- Pengujian tahap rilis
- Pengujian pengguna

Tujuan Pengujian Program

- Untuk menunjukkan kepada pelanggan dan pengembang bahwa PL sudah sesuai dengan kebutuhan yang ditetapkan
  - Pengujian validasi
- Untuk mengetahui kondisi-kondisi yang membuat program menjadi salah, tidak diharapkan atau tidak berfungsi sebagaimana mestinya
  - Pengujian kecacatan (kerusakan)

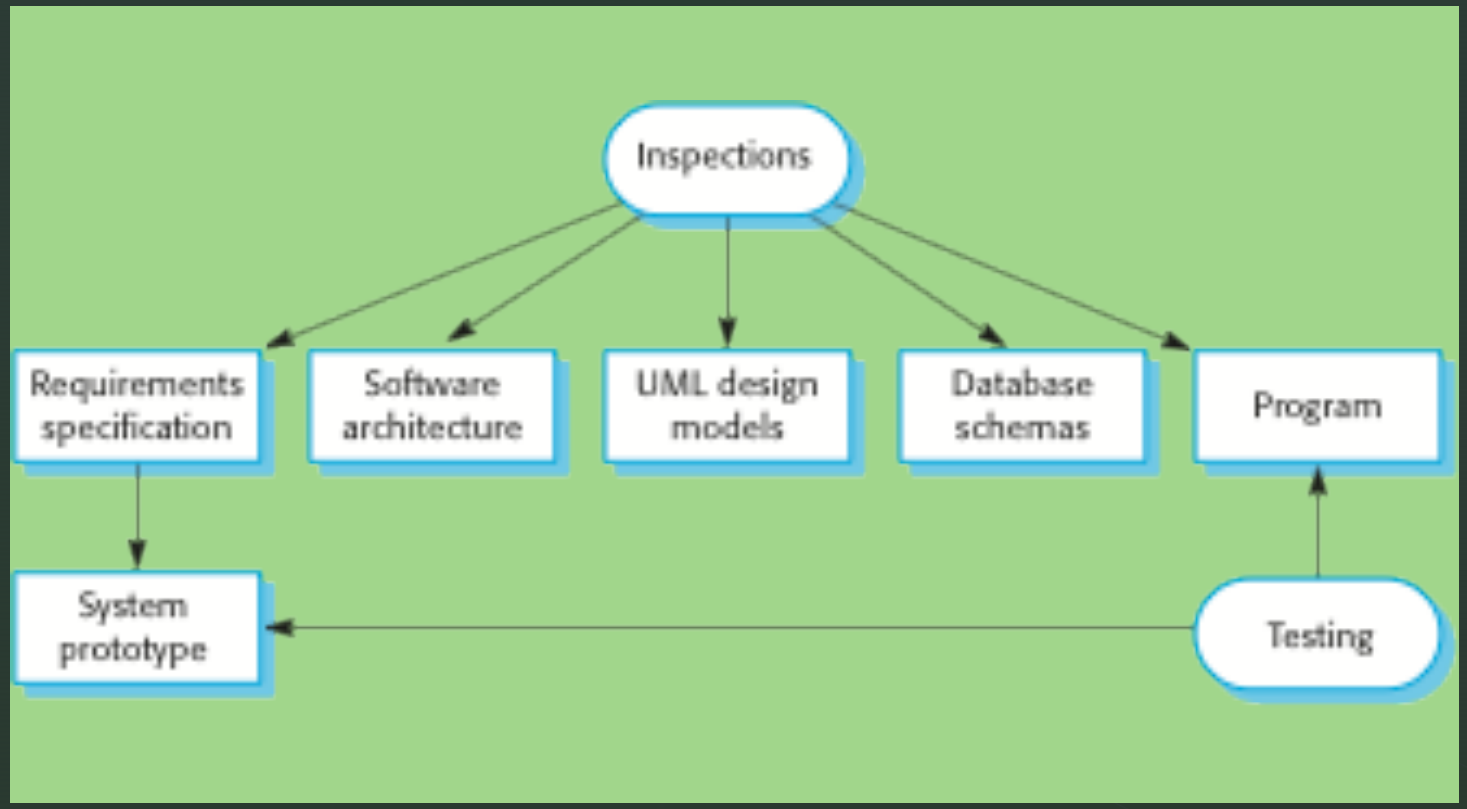
Verifikasi vs Validasi

- Verifikasi
  - Apakah PL telah dibangun dengan benar?
  - PL harus memenuhi semua spesifikasi kebutuhan yang telah ditentukan
- Validasi
  - Apakah PL yang dibangun merupakan produk yang benar?
  - PL harus berfungsi sesuai dengan kebutuhan nyata pelanggan

Inspeksi dan Pengujian PL

- Inspeksi PL
  - Analisis terhadap representasi sistem yang wujudnya statis untuk mencari permasalahan
  - Verifikasi statis
- Pengujian PL
  - Pengujian dan pengamatan perilaku PL
  - Sistem dieksekusi dengan data uji dan perilaku operasionalnya diamati
  - Verifikasi dinamis

Inspeksi dan Pengujian PL



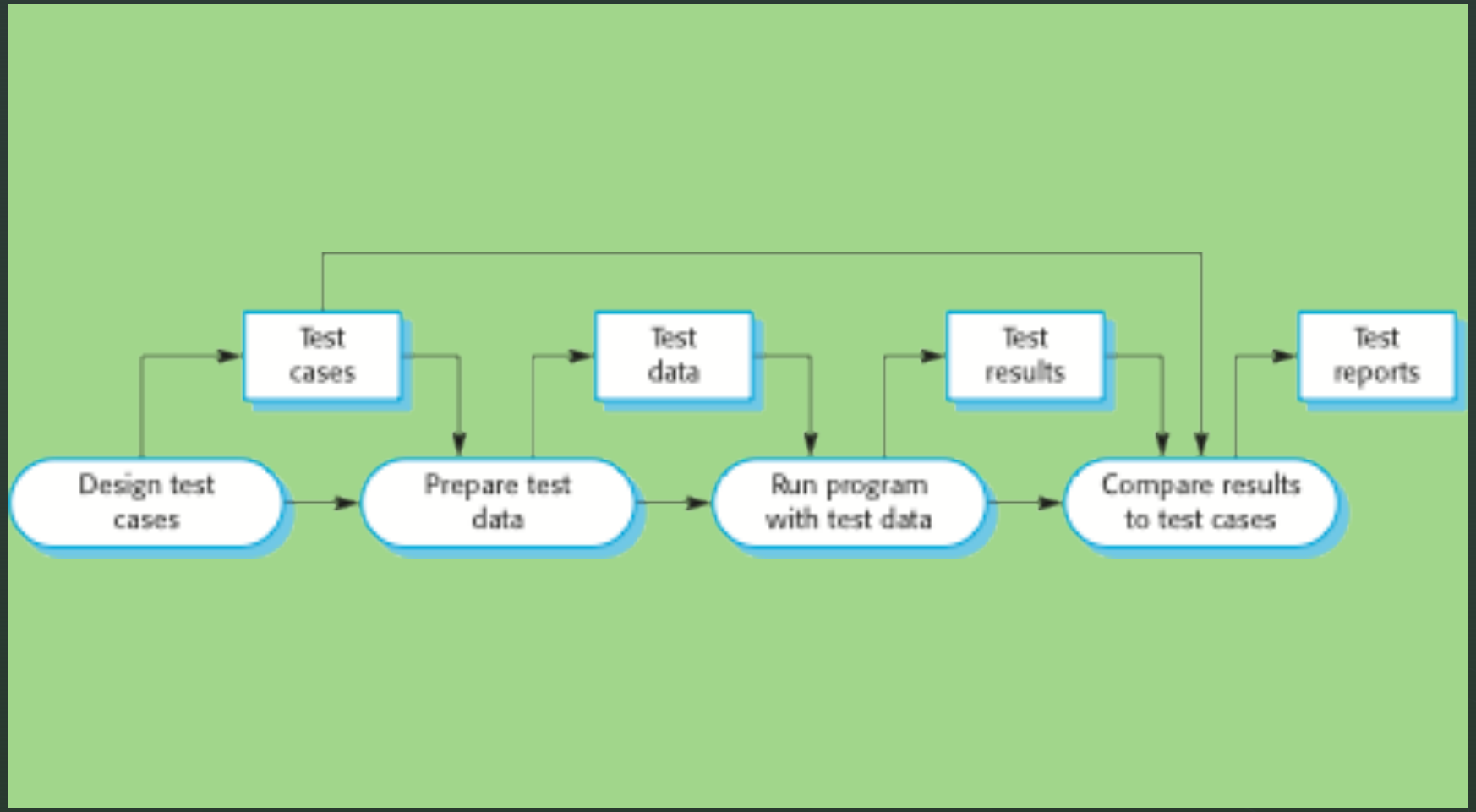
Inspeksi PL

- Sebuah tim inspeksi PL akan memeriksa kode sumber untuk mencari anomali dan cacat pada PL
- Proses inspeksi tidak memerlukan eksekusi program, sehingga dapat dilakukan sebelum tahap implementasi
- Proses inspeksi dapat diterapkan pada artifak statis lainnya, semisal dokumen spesifikasi PL, data konfigurasi PL, data uji, artifak perancangan, dll.
- Telah terbukti bahwa proses inspeksi merupakan teknik yg cukup efektif untuk mencari kesalahan pada program

Inspeksi dan pengujian PL

- Inspeksi dan Pengujian adalah proses yang saling melengkapi
- Keduanya sebaiknya dilakukan selama proses V & V
- Inspeksi dapat memeriksa kesesuaian dengan spesifikasi PL, namun tidak dapat memeriksa kesesuaian dengan kebutuhan nyata pelanggan
- Inspeksi tidak dapat memeriksa karakteristik non fungsional semisal performansi, kebergunaan, dsb

Model Proses Pengujian PL



Tahapan pengujian PL

- Pengujian tahap pengembangan
  - Pengujian selama pengembangan PL untuk mencari ‘bug’ dan cacat/rusak
- Pengujian tahap rilis
  - Tim penguji (bukan yang membuat program) melakukan pengujian secara menyeluruh terhadap PL versi lengkap sebelum PL dirilis kepada pengguna
- Pengujian pengguna
  - Pengguna atau calon potensial pengguna menguji sistem di lingkungan tempat sistem dioperasikan

Pengujian Tahap Pengembangan Perangkat Lunak

Pengujian tahap pengembangan

- Jenisnya:
  - Pengujian unit
    - Pengujian pada unit program yang terkecil (kelas, modul)
    - Fokus pada pengujian fungsionalitas unit
  - Pengujian komponen
    - Pengujian pada sekelompok unit program yang diintegrasikan membentuk komponen
    - Fokus pada pengujian antarmuka komponen
  - Pengujian sistem
    - Pengujian pada sekelompok komponen yang diintegrasikan membentuk satu sistem yang lengkap
    - Fokus pada pengujian interaksi antar komponen

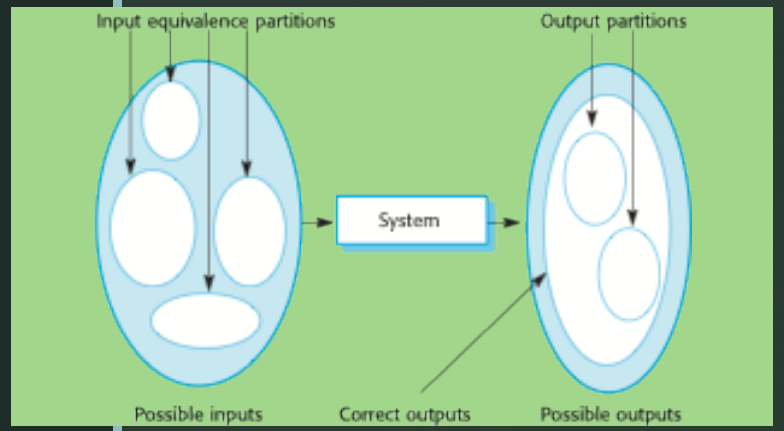
Strategi Pengujian

- Pengujian partisi
  - mengidentifikasi sekelompok input yang memiliki karakteristik sama dan harus diproses dengan cara yang sama
- Pengujian berbasis pedoman (*guideline*)
  - Menggunakan pedoman untuk memilih kasus uji
  - Pedoman tsb umumnya berisi pengalaman terdahulu tentang kesalahan yang ditemui oleh pemrogram

Pengujian Partisi

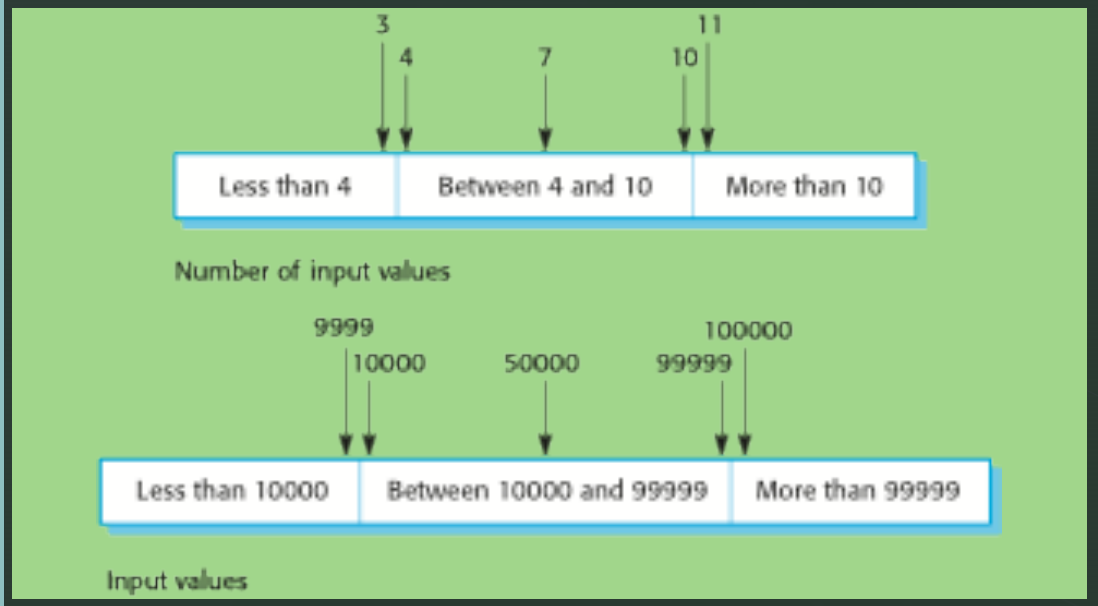
- Pengujian partisi
  - Teknik pengujian PL yang membagi input data ke dalam partisi-partisi data tempat kasus-kasus uji dapat diturunkan
  - Kasus uji dirancang agar dapat mencakup semua partisi setidaknya untuk sekali uji
  - Keuntungannya adalah mengurangi waktu pengujian karena kasus ujinya diminimalisir untuk setiap partisi
  - Partisi data disebut juga dengan partisi ekivalen

Pemartisian Ekivalen (Equivalence Partitioning)



- Partisi ekuivalensi adalah metode yang membagi domain input dari suatu program ke dalam kelas data, menentukan kasus pengujian dengan mengungkapkan kelas-kelas kesalahan, sehingga akan mengurangi jumlah keseluruhan kasus pengujian. Bila suatu *link weight* mempunyai pola *transitivitas*, *simetris*, dan *refleksif* maka akan terdapat kelas ekuivalensi. Kelas ekuivalensi merepresentasikan serangkaian kondisi valid dan invalid untuk kondisi inputan. Secara khusus, suatu kondisi input dapat berupa harga *numeric*, suatu rentang harga, serangkaian harga yang terkait, atau suatu kondisi *Boolean* (Perry, 1995). Penentuan Kelas Ekuivalensi:

Partisi-partisi ekivalen



Penentuan Kelas Ekuivalensi:

1. Bila kondisi input menentukan suatu range, maka satu kelas ekuvalensi valid dan dua yang invalid ditentukan.
2. Bila suatu kondisi input memerlukan suatu harga khusus, maka satu kelas ekuivalensi valid dan dua yang invalid ditentukan.
3. Bila suatu kondisi menentukan anggota suatu himpunan, maka satu kelas ekuivalensi valid atau dua yang invalid ditentukan.
4. Bila suatu kondisi input adalah *boolean*, maka satu kelas valid dan satu yang lain ditentukan

Contoh :
Nomor telepon untuk SLJJ Ketentuan :
Kode area : kosong atau ada 3 – 4 digit yang dimulai dengan 0
Prefiks : 3 digit dengan tidak dimulai dengan 1 atau 0
Sufiks : 4 digit

Kondisi input :

Kode area : kondisi input : Boolean – boleh ada boleh tidak ada

Kondisi input : range – nilai >200 atau nilai < 999

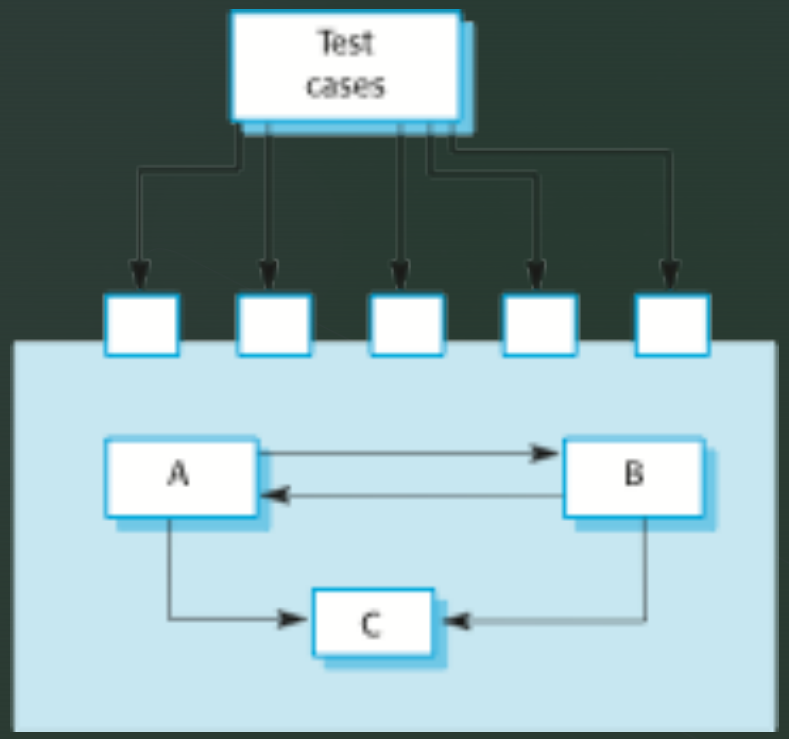
Prefiks : Kondisi input : range – nilai > 200 atau nilai < 999

Surfiks : kondisi input : harga – 4 digit

Pengujian berbasis pedoman (urutan)

- Pengujian dengan urutan yang menghasilkan nilai tunggal
- Menggunakan urutan yang berbeda ukuran pada setiap pengujian
- Kasus uji harus dibuat agar bisa mencakup pengujian pada elemen awal, tengah, dan elemen akhir
- Pengujian dengan urutan untuk kasus nol

Pengujian komponen



- Difokuskan pada pengujian yang memperlihatkan bahwa antarmuka komponen sudah berperilaku seperti yang diharapkan

Pengujian Antarmuka

- Untuk mengetahui kegagalan karena kesalahan pada antarmuka
  - kesalahan antarmuka
  - kesalahan asumsi pada antarmuka
- Jenis-jenis antarmuka
  - Antarmuka parameter
  - Antarmuka ‘shared memory’
  - Antarmuka prosedural
  - Antarmuka ‘message passing’

Pengujian Sistem

- Difokuskan pada pengujian interaksi antar komponen
- Melibatkan tim penguji yang terpisah dan tidak berhubungan dengan tim perancang dan pemrogram
- Sebagai dasar pengujian, dapat menggunakan kasus penggunaan (*use case*)
  - Pengujian kasus penggunaan

Pengujian Regresi

- Adalah pengujian sistem untuk memastikan bahwa ‘error’ tidak muncul di bagian lain dari PL karena adanya perubahan pada bagian tertentu
  - untuk memastikan bahwa ‘error’ tidak merambat ke tempat lain dari PL tsb

Referensi

- Pressman, R. S., *Software Engineering: A Practitioners Approach*, 7th edition, Pearson Education, 2009.
- Sommerville, I., *Software Engineering,* 9th edition, Addison-Wesley, 2010.